RouteLLM API Reference

RouteLLM provides an OpenAI-compatible API endpoint that intelligently routes your requests to the most appropriate underlying model based on cost, speed, and performance requirements.

Overview

RouteLLM is a smart routing layer that automatically selects the best model for your request, balancing performance, cost, and speed. Instead of manually choosing between different models, you can use the `route-llm` model identifier and let the system make the optimal choice for you.

Key Features

- **Intelligent Routing**: Automatically selects the best model based on request complexity
- **Cost Optimization**: Routes to cost-effective models when appropriate
- **Performance Tuning**: Uses high-performance models for complex tasks
- **Multimodal Support**: Supports text and image inputs for compatible models
- **PDF Support**: Process and analyze PDF documents as input for compatible models
- **Image Generation**: Generate high-quality images from text prompts using state-of-the-art models
- **Streaming Support**: Real-time response streaming available
- **Tool Calling**: Invoke functions from the model response and submit results back for multi-step workflows

Getting Started

How It Works

1. **Sign Up**: Sign up as a ChatLLM subscriber to access RouteLLM API
2. **Access the API**: Click on the RouteLLM API icon in the lower left corner of the ChatLLM interface to access API documentation and details
3. **Get Your API Key**: Obtain your API key from the RouteLLM API page

Ask AI

4. **Start Using**: Invoke the API for any LLM and use it in your applications

Why Choose RouteLLM API?

RouteLLM API comes with your ChatLLM subscription, providing several key benefits:

- **Unified Platform**: Use all LLMs (both open-weight and Proprietary) in the ChatLLM Teams UX and via API, all in one place
- **Easy Management**: Centralized way to manage all your favorite AI model consumption
- **Flexible Access**: Access models through both the user interface and programmatic API
- **Cost-Effective**: Competitive pricing with best available rates for open-source models
- **Transparent Pricing**: No markup on proprietary LLMs - you pay provider prices

Pricing

Credit System

The ChatLLM subscription includes **20,000 credits** to get you started. Each API call consumes credits proportional to the cost of the LLM call. RouteLLM is available for **unlimited use** for ChatLLM subscribers - while it still tracks credits for accounting purposes, you can continue to use RouteLLM even after hitting your monthly credit limit.

Pricing Details

Proprietary LLMs

Proprietary LLMs (e.g., OpenAI, Anthropic, Google Gemini, etc.) are priced based on the prices advertised by the provider. **We DO NOT charge you more than what the provider does.** Prices are updated automatically whenever the provider updates their pricing.

Open-Weight LLMs

Open-Weight LLMs are typically priced at the **best available price on the planet**. Our prices typically match the best available price anywhere in the world.

> **Note**: All open weight LLMs are hosted on servers based in the United States.

View Current Pricing

Pricing for each LLM is published in our RouteLLM API documentation. You can also use the `listRouteLLMModels` endpoint to programmatically retrieve the most up-to-date list of available models and their current pricing.

Base URLs

The base URL depends on your organization type:

- **Self-Serve Organizations**: `https://routellm.abacus.ai/v1`
- **Enterprise Platform**: `https://<workspace>.abacus.ai/v1`

Replace `<workspace>` with your specific workspace identifier for enterprise deployments. To know your correct base url, refer: RouteLLM API.

Authentication

All API requests require authentication using an API key. Include your API key in the request header:

```
Authorization: Bearer <your_api_key>
```

You can obtain your API key from the Abacus.AI platform.

Supported Models

The RouteLLM API supports a wide range of models for both text generation and image generation. You can specify a model explicitly or use `route-llm` to let the system decide.

Routing Model

- **`route-llm`**: Intelligently routes to one of Claude 4.5 Sonnet, GPT-5.2, or Gemini 3 Flash based on the complexity of the request. This is the recommended option for most use cases.

Text Generation Models

You can also directly target specific text generation models. The following models are currently supported:

OpenAI Models

- `gpt-5.2`, `gpt-5.1`
- `gpt-5`, `gpt-5-mini`, `gpt-5-nano`
- `gpt-4.1`, `gpt-4.1-mini`, `gpt-4.1-nano`
- `gpt-4o`, `gpt-4o-mini`
- `o4-mini`, `o4-mini-high`
- `o3`, `o3-high`, `o3-mini`, `o3-pro`
- `o1`, `o1-mini`

Anthropic Models

- `claude-sonnet-4-6`, `claude-opus-4-6`
- `claude-4-5-sonnet`, `claude-4-5-haiku`, `claude-4-5-opus`
- `claude-sonnet-4`, `claude-opus-4`, `claude-opus-4-1`
- `claude-3-7-sonnet`

Google Models

- `gemini-3.1-pro`, `gemini-3-pro`, `gemini-3-flash`
- `gemini-2.5-flash`, `gemini-2.5-pro`

xAI Models

- `grok-4-1-fast`, `grok-4`, `grok-code-fast-1`

Meta Models

- `llama-4-Maverick-17B`
- `llama-3.1-405B`, `llama-3.3-70B`

DeepSeek Models

- `deepseek-v3.2`, `deepseek-v3.1`, `deepseek-v3.1-Terminus`

- `deepseek-R1`

Qwen Models

- `qwen3-max`, `qwen3-coder`
- `qwen3-32b`, `qwq-32b`

Kimi Models

- `kimi-k2`, `kimi-k2.5`

ZhipuAI GLM Models

- `glm-4.5`, `glm-4.6`, `glm-4.7`, `glm-5`

> **Note**: This list is subject to change as new models are added. Use the `listRouteLLMModels` endpoint to get the most up-to-date list of available models and their pricing.

Image Generation Models

For image generation, the following models are supported:

- **`flux-2-pro`**: FLUX-2 PRO - High-quality, photorealistic image generation
- **`flux-kontext`**: FLUX Kontext - Advanced image generation
- **`dall-e`**: OpenAI DALL-E - High-quality creative image generation
- **`ideogram`**: Ideogram - Excellent for text rendering in images
- **`recraft`**: Recraft - Design and illustration focused
- **`imagen`**: Google Imagen - Image generation
- **`nano-banana-pro`**: Nano Banana Pro - High-quality image generation
- **`seedream`**: Seedream 4.5 - Image generation model

Request Parameters

1. Required Parameters

`messages` (array, required)

A list of messages comprising the conversation so far. Each message must be an object with the following structure:

- `role` (string, required): The role of the message sender. Must be one of:
  - `user`: Messages from the user/end-user
  - `assistant`: Previous responses from the AI assistant
  - `system`: System-level instructions that guide the assistant's behavior
- `content` (string or array, required): The content of the message. Can be:
  - A string for text-only messages
  - An array for multimodal content (text and images)

2. Optional Parameters

`model` (string, optional)

The ID of the model to use. Can be either a text generation model or an image generation model, depending on the `modalities` parameter. If omitted, defaults to `route-llm`.

Text Generation Models: `route-llm`, `gpt-5.1`, `claude-4-5-sonnet`, `gemini-2.5-pro`, etc.

Image Generation Models: `flux-2-pro`, `flux-kontext`, `dall-e`, `ideogram`, `recraft`, `imagen`, `nano-banana-pro`, `seedream`

Examples: `route-llm`, `gpt-5.1`, `flux-2-pro`, `seedream`

`max_tokens` (integer, optional)

The maximum number of tokens to generate in the chat completion. The total length of input tokens and generated tokens is limited by the model's context window.

Default: Model-dependent

`temperature` (number, optional)

What sampling temperature to use, between 0 and 2. Higher values like 0.8 will make the output more random, while lower values like 0.2 will make it more focused and deterministic.

Default: 1.0

Recommended values:

- `0.0-0.3`: For factual, deterministic responses
- `0.7-1.0`: For creative, varied responses
- `1.0-2.0`: For highly creative, diverse outputs

`top_p` (number, optional)

An alternative to sampling with temperature, called nucleus sampling, where the model considers the results of the tokens with top_p probability mass. So 0.1 means only the tokens comprising the top 10% probability mass are considered.

Default: 1.0

Range: 0.0 to 1.0

`stream` (boolean, optional)

If set to `true`, partial message deltas will be sent as data-only server-sent events as they become available. The stream will terminate by a `data: [DONE]` message.

Default: `false`

`stop` (string or array, optional)

Up to 4 sequences where the API will stop generating further tokens. The returned text will not contain the stop sequence.

Example: `stop": ["Human:", "AI:"]`

`presence_penalty` (number, optional)

Number between -2.0 and 2.0. Positive values penalize new tokens based on whether they appear in the text so far, increasing the model's likelihood to talk about new topics.

Default: 0.0

`frequency_penalty` (number, optional)

Number between -2.0 and 2.0. Positive values penalize new tokens based on their existing frequency in the text so far, decreasing the model's likelihood to repeat the same line verbatim.

Default: 0.0

`response_format` (object, optional)

An object specifying the format that the model must output. Two types are supported:

1. JSON Object Mode

```
"response_format": {
  "type": "json_object"
}
```

Constrains the model to output valid JSON. You must also instruct the model to produce JSON via a system or user message.

2. JSON Schema Mode

```
"response_format": {
  "type": "json_schema",
  "json_schema": {
    "name": "your_schema_name",
    "schema": {
      "type": "object",
      "properties": {
        "field_name": { "type": "string" },
        "count": { "type": "integer" }
      },
      "required": ["field_name", "count"],
      "additionalProperties": false
    }
  }
}
```

JSON Schema mode constrains the model to output JSON that strictly conforms to the provided schema. No system or user message instructing the model to produce JSON is required — the

schema itself enforces the format. The `json_schema` object requires:

Field	Type	Required	Description
`name`	string	Yes	A name identifier for the schema
`schema`	object	Yes	The JSON Schema definition
`strict`	boolean	No	Whether to enforce strict schema adherence (see below)

The inner `schema` object requires:

Field	Type	Required	Description
`type`	string	Yes	JSON Schema type (e.g., `"object"`)
`properties`	object	Yes	Property definitions for the object
`required`	array	Yes	List of required property names
`additionalProperties`	boolean	Yes	Whether to allow extra properties beyond those defined

`strict` mode

The `strict` field controls how rigidly the model follows the schema:

- `strict: true` — The schema is treated as a **law**. The model is guaranteed to produce output that exactly matches the schema. Every field in `required` will be present, no extra fields are added, and types are enforced precisely.
- `strict: false` (default) — The schema is treated as a **suggestion**. The model will try to follow it, but may deviate in edge cases (e.g., omitting optional fields or adding extra context).

Use `strict: true` whenever your downstream code parses the response programmatically.

> **Important**: When using `response_format: { type: "json_object" }`, you must instruct the model to produce JSON via a system or user message. This is **not required** for `json_schema` mode — the schema enforces the format automatically.

`tools` (array, optional)

A list of tools the model may call. Each tool is an object with:

- `type`: Must be `"function"`.
- `function`: Object with:
  - `name` (string, required): Name of the function the model can call.
  - `description` (string, optional): Description of the function for the model.
  - `parameters` (object, optional): JSON Schema for the function parameters (OpenAI-style).

Example:

```
"tools": [
  {
    "type": "function",
    "function": {
      "name": "get_current_weather",
      "description": "Get the current weather in a given location",
      "parameters": {
        "type": "object",
        "properties": {
          "location": { "type": "string", "description": "City and state,
e.g. San Francisco, CA" },
          "unit": { "type": "string", "enum": ["celsius", "fahrenheit"] }
        },
        "required": ["location"]
      }
    }
  }
]
```

`tool_choice` (string or object, optional)

Controls whether the model can call tools. Values:

- `"none"`: Do not call any tool (default when `tools` is omitted).

- `"auto"`: Model may choose to call one or more tools (default when `tools` is provided).
- `{"type": "function", "function": {"name": "get_current_weather"}}`: Force the model to call the named function.

Default: `"auto"` when `tools` is provided.

Response Format

Non-Streaming Response Streaming Response

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1677858242,
  "model": "route-llm",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "The meaning of life is..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 20,
    "total_tokens": 30
  }
}
```

Response Fields

- `id`: A unique identifier for the chat completion
- `object`: The object type, always `chat.completion` (or `chat.completion.chunk` for streaming)
- `created`: The Unix timestamp of when the completion was created

- `model`: The model used for the completion (may differ from the requested model if using `route-llm`)
- `choices`: A list of completion choices
  - `index`: The index of the choice
  - `message`: The message object (non-streaming) or `delta` (streaming)
  - `finish_reason`: The reason the completion finished (`stop`, `length`, `content_filter`, `tool_calls`, or `null` for streaming)
- `usage`: Token usage statistics (not present in streaming responses until the final chunk)

Tool Calling

The API supports tool (function) calling: the model can request that your application run a function and return the result in a follow-up request. This enables multi-step workflows (e.g. get weather, query a database, run code).

> NOTE
>
> **Currently, tool calling is stateless.** The server does not execute tools or persist tool-call state. Your application must run the requested functions, send the results back in a follow-up request (with the same `tools` and full message history), and handle any multi-step flow on the client side.

Request: Defining tools

Pass a `tools` array with one or more functions. Optionally set `tool_choice` to `"auto"` (default), `"none"`, or a specific function to force.

Example request with tools:

```
{
  "model": "route-llm",
  "messages": [
    {"role": "user", "content": "What's the weather in Boston?"}
  ],
  "tools": [
    {
      "type": "function",
```

```
      "function": {
        "name": "get_current_weather",
        "description": "Get the current weather in a given location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": { "type": "string", "description": "City and state"
},
            "unit": { "type": "string", "enum": ["celsius", "fahrenheit"] }
          },
          "required": ["location"]
        }
      }
    }
  ],
  "tool_choice": "auto"
}
```

Response: model requests a tool call

When the model decides to call a tool, the completion message includes a `tool_calls` array and `finish_reason` is `"tool_calls"`. The message `content` may be empty or contain reasoning.

Non-Streaming Response Streaming Response

Example response with tool_calls:

```
{
  "id": "chatcmpl-xyz",
  "object": "chat.completion",
  "created": 1677858242,
  "model": "route-llm",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": null,
        "tool_calls": [
          {
```

```
            "id": "call_abc123",
            "type": "function",
            "function": {
              "name": "get_current_weather",
              "arguments": "{\"location\": \"Boston, MA\", \"unit\":
\"fahrenheit\"}"
            }
          }
        ]
      },
      "finish_reason": "tool_calls"
    }
  ],
  "usage": { "prompt_tokens": 20, "completion_tokens": 25, "total_tokens":
45 }
}
```

Follow-up: sending tool results

To continue the conversation, send the assistant message (including `tool_calls`) and add a message with `role: "tool"` for each tool call, providing the `tool_call_id` and the result as `content`.

Example follow-up request:

```
{
  "model": "route-llm",
  "messages": [
    {"role": "user", "content": "What's the weather in Boston?"},
    {
      "role": "assistant",
      "content": null,
      "tool_calls": [
        {
          "id": "call_abc123",
          "type": "function",
          "function": {
            "name": "get_current_weather",
            "arguments": "{\"location\": \"Boston, MA\", \"unit\":
\"fahrenheit\"}"
          }
        }
      ]
```

```
    },
    {
      "role": "tool",
      "tool_call_id": "call_abc123",
      "content": "{\"temperature\": 72, \"unit\": \"fahrenheit\",
\"conditions\": \"Sunny\"}"
    }
  ],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "get_current_weather",
        "description": "Get the current weather in a given location",
        "parameters": {
          "type": "object",
          "properties": {
            "location": { "type": "string" },
            "unit": { "type": "string", "enum": ["celsius", "fahrenheit"] }
          },
          "required": ["location"]
        }
      }
    }
  ]
}
```

The model will then generate a final reply (e.g. summarizing the weather). Repeat the flow if it returns more `tool_calls`.

Notes:

- Include the same `tools` (and optionally `tool_choice`) in follow-up requests when continuing a tool-calling conversation.
- When streaming, tool call arguments arrive in multiple chunks. Use the `index` field to match chunks to specific tool calls, and concatenate the `arguments` strings before parsing as JSON.
- Multiple tool calls can be returned in a single response. In streaming mode, track each tool call by its `index` and aggregate separately.

PDF Support

PDF documents are supported as input for compatible models. Use the `file` content type with a `file` object (`filename`, `file_data`) for parsing.

Request schema:

```
{
  "model": "gpt-5.1",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "What are the main points in this document?"
        },
        {
          "type": "file",
          "file": {
            "filename": "document.pdf",
            "file_data": "https://bitcoin.org/bitcoin.pdf"
          }
        }
      ]
    }
  ]
}
```

Notes:

- Use `type: "file"` with a `file` object containing `filename` and `file_data`
- `file_data` can be an HTTPS URL to the PDF or base64-encoded content

Multimodal capabilities

The API supports multimodal inputs & outputs for models that support vision capabilities.

Image Analysis

Images can be provided in the two following ways as input:

Image via HTTPS URL Image via Base64

```
{
  "model": "route-llm",
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "Describe the image"},
        {
          "type": "image_url",
          "image_url": {
            "url": "https://example.com/image.jpg"
          }
        }
      ]
    }
  ]
}
```

Image Support Notes:

- Supported formats: PNG, JPEG, WebP, and GIF
- Images are automatically resized and processed by the API
- Multiple images can be included in a single message
- Base64 images should use the data URI format: `data:image/<format>base64,<base64_string>`

Image Generation

The RouteLLM API supports image generation from text prompts using state-of-the-art image generation models. Image generation uses the **unified chat completions endpoint** with the `modalities` and `image_config` parameters.

> **Note**: In addition to dedicated image generation models (e.g., `flux-2-pro`, `seedream`, `ideogram`), Gemini and OpenAI models also support image generation when used with the `modalities: ["image"]` parameter.

Request Syntax

Image generation uses the same unified schema as text generation, with additional parameters for image generation:

```
{
  "model": "string (required)",
  "messages": [
    {
      "role": "user",
      "content": "string (required)"
    }
  ],
  "modalities": ["image"],
  "image_config": {
    "num_images": "integer (optional)",
    "aspect_ratio": "string (optional)",
    ...
  }
}
```

Request Parameters

`model` (string, required)

The ID of the model to use for image generation. Can be any supported image generation model or a Gemini/OpenAI model that supports image generation.

Supported Models:

- **Dedicated Image Generation Models**: `flux-2-pro`, `flux-kontext`, `seedream`, `ideogram`, `recraft`, `imagen`, `nano-banana-pro`, `dall-e`
- **Gemini Models** (support image generation): `gemini-2.5-pro`, `gemini-2.5-flash`, `gemini-3-pro`, `gemini-3-flash`
- **OpenAI Models** (support image generation): `gpt-5.1`, `gpt-5.2`, `gpt-5`, `gpt-4o`, etc.

Examples: `flux-2-pro`, `seedream`, `gemini-2.5-pro`, `gpt-5.1`

`messages` (array, required)

A list of messages comprising the conversation. The user's message should contain the prompt for image generation.

Example:

```
{
  "role": "user",
  "content": "A beautiful sunset over mountains"
}
```

`modalities` (array, optional)

Specifies what type of content to generate.

Valid values:

- `["image"]`: Generate images
- `["text"]`: Generate text (default if not specified)

Default: `["text"]` (if not specified)

> **Note**: You can generate either images or text in a single request, not both simultaneously.

`image_config` (object, optional)

Configuration object for image generation. Required when `modalities` includes `image`.

> **Important**:
>
> - `num_images` is supported by all image generation models
> - `aspect_ratio` is supported by all image generation models
> - `image_size` is only supported by OpenAI & Gemini models
> - `resolution` is only supported by OpenAI & Gemini models
> - `quality` is only supported by OpenAI & Gemini models

Image Config properties

All Models OpenAI & Gemini Only

Parameter	Type	Description	Valid Values / Range	Default	Example
`num_images`	`integer`, optional	The number of images to generate. Supported by all image generation models.	`1–4`	`1`	`3`
`aspect_ratio`	`string`, optional	The aspect ratio of the generated images. Supported by all image generation models.	Model-dependent (Ex. `1:1`, `2:3`, `3:4`, `9:16`)	Model-dependent (typically `1:1`)	`2:3`

Model-Specific Configurations

Different image generation models have unique strengths and support different parameters:

Model	Best For	Supported Parameters	Supported Aspect Ratios
FLUX-2 PRO `flux-2-pro`	Photorealistic images, high-quality portraits, detailed scenes	`num_images`, `aspect_ratio`	`1:1`, `3:4`, `4:3`, `16:9`, `9:16`
FLUX Kontext `flux-kontext`	Context-aware image generation, complex scenes	`num_images`, `aspect_ratio`	`1:1`, `2:3`, `3:2`, `3:4`, `4:3`, `16:9`,

Model	Best For	Supported Parameters	Supported Aspect Ratios
			`9:16`, `9:21`, `21:9`
DALL-E `dalle`	Creative and artistic images, safe content generation	`num_images`, `aspect_ratio`, `quality`, `resolution`, `image_size`	`1:1`, `2:3`, `3:2`, `16:9`, `9:16`
Ideogram `ideogram`	Text rendering in images, typography, logos	`num_images`, `aspect_ratio`	`1:1`, `2:3`, `3:2`, `3:4`, `4:3`, `16:9`, `9:16`, `10:16`, `16:10`
Recraft `recraft`	Design and illustration work, vector-style images	`num_images`, `image_size`	`1024x1024`, `1024x2048`, `2048x1024`, `1024x1365`, `1365x1024`
Google Imagen `imagen`	General-purpose image generation	`num_images`, `aspect_ratio`	`1:1`, `3:4`, `4:3`, `16:9`, `9:16`
Nano Banana Pro `nano-banana-pro`	High-quality artistic images	`num_images`, `aspect_ratio`	`1:1`, `2:3`, `3:2`, `3:4`, `4:3`, `16:9`, `9:16`, `21:9`
Seedream `seedream`	General image generation	`num_images`, `aspect_ratio`	`1:1`, `4:3`, `3:4`, `16:9`, `9:16`
Gemini Models `gemini-2.5-`	General-purpose image generation	`num_images`, `aspect_ratio`,	`1:1`, `2:3`, `3:2`, `3:4`, `4:3`, `16:9`,

Model	Best For	Supported Parameters	Supported Aspect Ratios
`pro`, `gemini-3-pro`, etc.	with advanced configuration	`quality`, `resolution`, `image_size`	`9:16`
OpenAI Models `gpt-5.1`, `gpt-5.2`, etc.	High-quality image generation with advanced configuration	`num_images`, `aspect_ratio`, `quality`, `resolution`, `image_size`	`1:1`, `2:3`, `3:2`

Code Examples

1. Basic Image Generation

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

# Basic image generation
response = client.chat.completions.create(
    model="gemini-2.5-pro",
    messages=[
        {
            "role": "user",
            "content": "A beautiful sunset over mountains"
        }
    ],
    modalities=["image"],
    image_config={
        "num_images": 1
    }
```

```
)

# Extract image URLs from response
for content_item in response.choices[0].message.content:
    if content_item.type == "image_url":
        print(f"Generated image: {content_item.image_url.url}")
```

2. Multiple Images

Python SDK TypeScript/JavaScript cURL

```
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

# Generate multiple images
response = client.chat.completions.create(
    model="flux-2-pro",
    messages=[
        {
            "role": "user",
            "content": "A futuristic cityscape at night with neon lights and
flying cars"
        }
    ],
    modalities=["image"],
    image_config={
        "num_images": 3,
        "aspect_ratio": "1:1"
    }
)

# Extract all image URLs
image_urls = [
    item.image_url.url
    for item in response.choices[0].message.content
    if item.type == "image_url"
]
```

```
for idx, url in enumerate(image_urls, 1):
    print(f"Image {idx}: {url}")
```

3. Portrait Orientation

Python SDK TypeScript/JavaScript cURL

```
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

# Generate portrait-oriented image
response = client.chat.completions.create(
    model="flux-2-pro",
    messages=[
        {
            "role": "user",
            "content": "A full-body portrait of a fashion model in elegant
evening wear"
        }
    ],
    modalities=["image"],
    image_config={
        "num_images": 1,
        "aspect_ratio": "2:3"
    }
)

for content_item in response.choices[0].message.content:
    if content_item.type == "image_url":
        print(f"Portrait image: {content_item.image_url.url}")
```

4. OpenAI Model with Quality

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

# Generate high-quality image with OpenAI model
response = client.chat.completions.create(
    model="gpt-5.1",
    messages=[
        {
            "role": "user",
            "content": "A whimsical illustration of a magical forest with glowing mushrooms"
        }
    ],
    modalities=["image"],
    image_config={
        "num_images": 1,
        "aspect_ratio": "1:1",
        "quality": "high"
    }
)

for content_item in response.choices[0].message.content:
    if content_item.type == "image_url":
        print(f"Image URL: {content_item.image_url.url}")
```

5. Gemini Model with Image Size and Resolution

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

# Generate image using Gemini model with advanced parameters
response = client.chat.completions.create(
```

```
    model="gemini-2.5-pro",
    messages=[
        {
            "role": "user",
            "content": "A professional headshot of a business executive"
        }
    ],
    modalities=["image"],
    image_config={
        "num_images": 1,
        "aspect_ratio": "2:3",
        "image_size": "1024x1536",
        "resolution": "2K"
    }
)

for content_item in response.choices[0].message.content:
    if content_item.type == "image_url":
        print(f"Image URL: {content_item.image_url.url}")
```

> **Note**: The `image_config` parameters `resolution`, `image_size`, and `quality` are only supported for Gemini and OpenAI models. `num_images` and `aspect_ratio` are supported by all image generation models.

Response Schema

Image generation responses follow the same unified chat completion response format. When `modalities` includes `image`, the response will contain image data in addition to any text content.

Success Response (Image Only)

```
{
  "created": 1677858242,
  "model": "gemini-2.5-pro",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": '',
        "images": [
```

```
          {
            "type": "image_url",
            "image_url": {
              "url": "https://example.com/generated-image-1.png"
            }
          },
          {
            "type": "image_url",
            "image_url": {
              "url": "https://example.com/generated-image-2.png"
            }
          }
        ]
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "compute_points_used": 150
  }
}
```

Error Handling

The API uses standard HTTP status codes to indicate success or failure:

- **200 OK**: Request succeeded
- **400 Bad Request**: Invalid request (missing parameters, invalid format, etc.)
- **401 Unauthorized**: Missing or invalid API key
- **429 Too Many Requests**: Rate limit exceeded
- **500 Internal Server Error**: Server error

Error Response Format

```
{
  "error": {
    "message": "The 'messages' parameter is missing, empty, or not a list.",
    "type": "ValidationError",
    "code": "invalid_request_error"
```

```
  }
}
```

Common error scenarios:

- Missing required `messages` parameter
- Empty `messages` array
- Missing `role` or `content` in message objects
- Invalid `role` value (must be "user", "assistant", or "system")
- Invalid model name
- Rate limit exceeded

Code Examples

Basic Request

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

response = client.chat.completions.create(
    model="route-llm",
    messages=[
        {"role": "user", "content": "What is the meaning of life?"}
    ]
)

print(response.choices[0].message.content)
```

Streaming Request

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

stream = client.chat.completions.create(
    model="route-llm",
    messages=[
        {"role": "user", "content": "Explain quantum computing in simple terms."}
    ],
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
```

Conversation with History

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "My name is Alice."},
    {"role": "assistant", "content": "Nice to meet you, Alice! How can I help you today?"},
    {"role": "user", "content": "What's my name?"}
]
```

```
response = client.chat.completions.create(
    model="route-llm",
    messages=messages,
    temperature=0.7,
    max_tokens=150
)

print(response.choices[0].message.content)
```

JSON Mode

Python SDK | TypeScript/JavaScript | cURL

```
from openai import OpenAI
import json

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

response = client.chat.completions.create(
    model="route-llm",
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant that outputs JSON."
        },
        {
            "role": "user",
            "content": "Return a JSON object with keys 'name', 'age', and
'city'."
        }
    ],
    response_format={"type": "json_object"},
    temperature=0.7
)

content = response.choices[0].message.content
data = json.loads(content)
print(data)
```

Structured Output (JSON Schema)

Python SDK TypeScript/JavaScript cURL

```python
from openai import OpenAI
import json

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

response = client.chat.completions.create(
    model="route-llm",
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant that outputs JSON."
        },
        {
            "role": "user",
            "content": "Extract the name, age, and city from: 'Alice is 30 years old and lives in Paris.'"
        }
    ],
    response_format={
        "type": "json_schema",
        "json_schema": {
            "name": "person_info",
            "schema": {
                "type": "object",
                "properties": {
                    "name": {"type": "string"},
                    "age": {"type": "integer"},
                    "city": {"type": "string"}
                },
                "required": ["name", "age", "city"],
                "additionalProperties": False
            }
        }
    }
)
```

```
data = json.loads(response.choices[0].message.content)
print(data)  # {"name": "Alice", "age": 30, "city": "Paris"}
```

With Optional Parameters

Python SDK | TypeScript/JavaScript | cURL

```
from openai import OpenAI

client = OpenAI(
    base_url="<your base url>",
    api_key="<your_api_key>",
)

response = client.chat.completions.create(
    model="route-llm",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Write a haiku about programming."}
    ],
    max_tokens=100,
    temperature=0.8,
    top_p=0.9
)

print(response.choices[0].message.content)
```

Best Practices

1. **Use `route-llm` for most cases**: Let the system choose the optimal model automatically
2. **Include conversation history**: Provide full message history for better context
3. **Set appropriate `max_tokens`**: Prevent unnecessarily long responses
4. **Use streaming for long responses**: Improve user experience with real-time output
5. **Handle errors gracefully**: Implement retry logic for transient errors